Traceable scope, security and acceptance criteria | v0.2

Atlas - Requirements Document

Version: 0.2.0 (APPROVED TARGET SCOPE; IMPLEMENTATION IN PROGRESS)

Date: 2026-10-14 | Repository: Swayamjain1240/Atlas | Product: Local-First Personal Life Operating System

1. Purpose, actors and constraints

- Primary user: a single local device owner who explicitly chooses what personal data Atlas may index.
- Primary outcome: ask questions about indexed personal content and receive relevant, source-grounded answers without cloud inference.
- Requirement labels: CRITICAL, HIGH, MEDIUM and FUTURE. Acceptance requires an executable check, not a README claim.
- Current GitHub main is a v0.1 scaffold. These v0.2 requirements document the target, not current functional completion.

2. Functional requirements - querying

- FR-001 CRITICAL: Natural-language query over indexed content through React chat -> Express -> Python RAG.
- FR-002 CRITICAL: Answers include trustworthy citations and relevance scores linking back to indexed source/chunk.
- FR-003 HIGH: Multi-turn sessions can persist in SQLite with explicit retention controls.
- FR-004 HIGH: User filters by approved source types such as file or browser.
- FR-005 MEDIUM: Direct semantic search works without Ollama generation.
- FR-006 HIGH: If no relevant evidence exists, Atlas states that clearly; it must not fabricate citations.
- FR-007 HIGH: Errors, slow inference and canceled queries produce visible recoverable states.

3. Functional requirements - ingestion

- FR-010 CRITICAL: Ingest text and code (.txt, .md, .py, .js, .json, .csv, .html, .css, .yaml/.yml).
- FR-011 HIGH: Extract text from supported PDFs; report unreadable/scanned files without silent success claims.
- FR-012 HIGH: Extract text from .docx while preserving source identity.
- FR-013 HIGH: Import local Chrome/Edge/Brave history metadata after explicit consent, respecting OS file locks.
- FR-014 CRITICAL: Split extracted content into ~1500 character chunks with tracked overlap and provenance.
- FR-015 MEDIUM: Configure per-user watched directories; default to no scanning without consent.
- FR-016 MEDIUM: Detect changed/removed files and perform idempotent re-indexing.
- FR-017 HIGH: Enforce permitted paths, file-size quotas, safe parsers and extension allowlists.
- FR-018 HIGH: Expose ingestion jobs, progress, skipped files, errors and retry controls.

4. Functional requirements - memory and administration

- FR-020 HIGH: View recently indexed memories with metadata and source links.
- FR-021 HIGH: Delete specific memories and associated vectors/caches safely.
- FR-022 MEDIUM: Show real memory totals, source counts and per-source health.
- FR-023 MEDIUM: Manually start and retry permitted ingestion jobs.
- FR-024 HIGH: Persist non-secret configuration and chat history in SQLite.
- FR-025 HIGH: Allow source consent to be granted, inspected and revoked.
- FR-026 MEDIUM: Backup and restore local SQLite and ChromaDB with integrity checks.

5. Frontend requirements

- UI-001 CRITICAL: Every authenticated screen fits within a viewport without page-level scrolling; contained panels may scroll.
- UI-002 HIGH: Long forms use step flows or bounded panels; lists are paginated and keyboard navigable.
- UI-003 HIGH: Dark space theme with indigo/purple accents; React + Vite + Tailwind CSS.
- UI-004 HIGH: Three.js displays actual memory nodes, links and inspectable source details, with non-WebGL fallback.
- UI-005 HIGH: GSAP animations are restrained, cleanly disposed and disabled/reduced per motion preference.
- UI-006 HIGH: Responsive desktop, tablet and mobile layouts with visible focus and readable contrast.
- UI-007 CRITICAL: Chat shows history, sending/loading states, citations, response errors and retry.
- UI-008 MEDIUM: Timeline reflects real ingestion/memory timestamps, not placeholder events.
- UI-009 MEDIUM: Dashboard statistics derive from actual API data, not hard-coded counts/status.
- UI-010 MEDIUM: Settings manage directories, model selection and local service readiness.
- UI-011 HIGH: Client never directly contacts ChromaDB, SQLite, Ollama or Python internal API.

6. Data and AI requirements

- DAT-001 CRITICAL: Persistent local ChromaDB vectors for all embedded chunks with stable IDs and metadata.
- DAT-002 HIGH: SQLite stores sources, memory metadata, ingestion jobs, sessions and settings; migrations are versioned.
- DAT-003 HIGH: Retrieval citations preserve title, path, source type, chunk index and available page/line.
- DAT-004 HIGH: Default chunk size 1500 chars with 200 overlap; tune only with test evidence.
- DAT-005 HIGH: Track access frequency, timestamps, hashes and deletion state.
- DAT-006 HIGH: Use local Sentence-Transformers all-MiniLM-L6-v2 (384 dimensions) and Ollama llama3.2:3b by default.
- DAT-007 HIGH: Models and index versions are tracked; embedding model changes trigger re-embedding.
- DAT-008 CRITICAL: Personal content never automatically leaves the device. No analytics, hosted models or background upload.
- DAT-009 MEDIUM: Persist conversation context only when enabled, with clear delete option.
- DAT-010 HIGH: Re-ingestion and delete reconcile SQLite records with Chroma vectors after partial failures.

Traceable scope, security and acceptance criteria | v0.2

7. Backend and integration requirements

- INT-001 CRITICAL: Node.js + Express is the ONLY user-facing backend gateway.
- INT-002 CRITICAL: Python AI worker runs locally, exposes internal endpoints only on loopback and requires a separate internal credential.
- INT-003 CRITICAL: Ollama local inference; fail gracefully if Ollama/model is unavailable.
- INT-004 CRITICAL: Sentence-Transformers local embedding, ChromaDB similarity search and grounded RAG.
- INT-005 HIGH: SQLite has one application owner (Node); ChromaDB has one owner (Python).
- INT-006 HIGH: All public routes versioned /api/v1 with stable validation and standard JSON errors.
- INT-007 HIGH: Startup verifies required env vars, models, database permissions and loopback binding.
- INT-008 FUTURE: Email IMAP, Slack/Discord/WhatsApp export ingestion and calendar connectors need separate scope approval.
- INT-009 FUTURE: Autonomous agents, MCP, cloud sync, Tauri and multi-user/team mode are explicitly deferred.

8. Non-functional requirements and performance

- NFR-001 CRITICAL: Local-first, private by default; once dependencies and model weights are installed, core inference operates offline.
- NFR-002 CRITICAL TARGET: p95 end-to-end query under 2 seconds on defined reference hardware; benchmark and report deviations.
- NFR-003 HIGH TARGET: vector search under 200ms for 100K indexed chunks/documents (test dataset must specify which).
- NFR-004 MEDIUM TARGET: under 500MB idle RAM excluding Ollama model process; total resident memory must also be reported.
- NFR-005 HIGH: Windows 10+, macOS 12+, Ubuntu 20.04+ support and platform-specific smoke tests.
- NFR-006 HIGH: Interruption-safe ingestion jobs, bounded concurrency, cancellation and restart reconciliation.
- NFR-007 HIGH: No silent data loss; backups, restoration and corruption behavior tested.
- NFR-008 HIGH: Log useful request IDs and timing without logging document bodies or secrets.
- PERF-001 TARGET: typical LLM generation under 5 seconds, hardware/model-dependent.
- PERF-002 TARGET: batch embedding under 100ms/100 documents and 100 text files/sec ingestion; measure and revise honestly.
- PERF-003 TARGET: less than 2GB additional process RAM during heavy ingestion, excluding Ollama model as explicitly recorded.
- PERF-004 TARGET: less than 1GB index/db storage per 100K documents on the defined benchmark dataset.

9. Security requirements (release gates)

- SEC-001 CRITICAL: Secrets only in environment or protected local secret store; never checked into source or bundled frontend.
- SEC-002 CRITICAL: Strict schemas, maximum lengths, request body caps and escaped/safe rendering.
- SEC-003 CRITICAL: Local API auth enforced on all protected routes; constant-time key comparison and rotation.

- SEC-004 CRITICAL: Restrict both Express and Python to loopback by default; reject unexpected Host/Origin values.
- SEC-005 HIGH: CORS allowlist with exact local origins; no wildcard + credentials.
- SEC-006 HIGH: Helmet-style security headers and restrictive CSP; no unsafe-inline in production where feasible.
- SEC-007 HIGH: Rate limits on protected endpoints, ingestion workloads and local worker requests.
- SEC-008 CRITICAL: Debug/docs disabled for packaged builds; safe error responses without stack traces or PII.
- SEC-009 HIGH: Block path traversal, symlink escapes, arbitrary system directory scanning and untrusted URLs.
- SEC-010 HIGH: Protect against malicious PDFs/docs, oversized uploads, archive bombs and unbounded queues.
- SEC-011 HIGH: Prevent stored XSS, prompt-injection privilege escalation and command execution from retrieved content.
- SEC-012 HIGH: Secret scanning before commit, including staged content/history when appropriate; gitignore .env and data.
- SEC-013 HIGH: npm audit, pip-audit, Bandit and pinned/locked reproducible dependency builds.
- SEC-014 HIGH: Read/write least privilege on local data, database files and log directories.
- SEC-015 HIGH: Authenticated internal Node-to-Python requests; Python never trusts unauthenticated localhost calls.
- SEC-016 HIGH: Timeouts, cancellation, resource quotas and bounded retries on internal requests.
- SEC-017 HIGH: No hidden network telemetry; bundle assets/fonts for offline production and audit egress.
- SEC-018 HIGH: User-authorized delete/revoke pathways, clear backups handling and secure retention policy.
- SEC-019 HIGH: SQLite schema migration integrity, backup before destructive migrations and tested rollback.
- SEC-020 HIGH: Do not store API secrets persistently in frontend localStorage without an explicit threat review.
- SEC-021 HIGH: CORS is not authentication; defend against browser-origin attacks on localhost.
- SEC-022 HIGH: If passwords are added in future, use Argon2id or bcrypt and never plaintext.
- SEC-023 HIGH: Avoid logging absolute private paths, document text or model prompts unless explicitly enabled and redacted.
- SEC-024 HIGH: Validate source access and deletion authorization consistently through the full call chain.
- SEC-025 HIGH: Threat-model and test each new connector/agent before enabling it.

10. Deployment and documentation

- DEP-001 HIGH: Document Python 3.11+, Node.js 20+, Ollama setup and model pull; no paid LLM API required.
- DEP-002 HIGH: npm install and npm run build produce reproducible frontend build.
- DEP-003 HIGH: Backend and Python can be started with documented development and production scripts.
- DEP-004 MEDIUM: Docker Compose optional for advanced users, only after local privacy checks.

- DEP-005 HIGH: README with troubleshooting, configuration, security, privacy and first-run setup.
- DEP-006 MEDIUM: Versioned releases and changelog once implementation milestones ship.

11. Acceptance scenarios

- ACC-01: Fresh install, approved folder with five text files; ingestion job finishes and /stats reports real counts.
- ACC-02: Query produces relevant answer and citations traceable to original files; missing evidence yields no fabricated facts.
- ACC-03: Delete memory removes SQLite metadata, Chroma vectors and visible timeline entry; restart confirms deletion.
- ACC-04: Unauthorized/wrong API key, off-origin requests and path-traversal inputs are rejected.
- ACC-05: Ollama unavailable yields labeled retrieval-only response/error, not a false success.
- ACC-06: Disable network after first setup; local ingestion/search/chat still work and no blocked font/CDN requests occur.
- ACC-07: Restore backup and reconcile indexed counts without user data loss.
- ACC-08: One-viewport desktop/mobile layouts, accessible navigation, reduced-motion, WebGL fallback.
- ACC-09: Full smoke, integration, E2E, security scan and benchmark evidence recorded before marking done.